

ASHISH KUMAR

EMAIL ID: 202403002@dau.ac.in

DHIRUBHAI AMBANI UNIVERSITY

PROJECT REPORT

Synapse Annual Fest Database

1. Introduction

1.1 Purpose

Synapse is an annual fest of DA-IICT which includes thousands of attendees, various events, artists, accommodations, etc.

It becomes difficult to keep track of data like, when and where will this event take place? How many people will be attending this event? Are there any more rooms available for accommodation? How many, which artist(s) will be present in a particular event, their contact details. What are the different types of tickets available? Platinum, Golden, silver (varying number of events included), or tickets including only 1 particular event, and their availability.

Hence it becomes essential to maintain a database for synapse.

- > It eliminates manual data handling.
- > Provide real-time monitoring of event schedules, venues and ticket status.
- > Enable rapid generation of reports for Synapse committee.
- > It avoids human errors, manual ticketing which often leads to errors.
- > Quick updates, last minute changes in venues or timing require rapid actions, which cannot be achieved by humans.

1.2 Intended audience

Database will be accessed by synapse committee members, administration, volunteers and IT staff.

-> Suppose they want to arrange food, water or beverages for a particular event, how much of these should they arrange? They can look for numbers of tickets sold for

this particular event to get an idea of size of audience. Once they have the idea of size of audience, they can also arrange for seating arrangement.

-> Suppose an artist is running late for an event, committee members can find his contact details quickly from the database and contact him for updates.

-> IT staff: tasked with system deployment, trouble shooting and ongoing maintenance. They need operational instructions, back up plans and support requirements.

1.3 Product scope

The System will provide the committee with tools

to:

- Create and manage events (music, dance, drama, tech, art, formal events etc.)
- Manage venues and schedule events to avoid clashes
- Register artists/performers and link them to events
- Support multiple ticket types and pass models (single-event, event-package, Platinum/Gold passes that cover multiple or all events)
- Record ticket sales details, buyer's details.
- Manage attendee information and accommodation bookings
- Maintain records for food vendors and sponsor details
- Generate standard reports (tickets sold per-event, revenue, artist schedule, capacity utilization)

Out of scope (for initial release):

- Full integration with live payment gateway
- Mobile native app.

1.4 Description

Synapse is the *flagship techno-cultural festival* of DAU (Dhirubhai Ambani University), held annually in Gandhinagar, Gujarat. It spans four electrifying days, typically in February, and brings together a vibrant fusion of *technology, culture, arts, and entertainment*.

Celebrating the intersection of creativity and innovation, Synapse features an extensive lineup—ranging from pro-nights with celebrated artists to competitions

across music, dance, drama, fashion, literature, photography, eSports, coding, and more. Such diversity makes it one of Gujarat's most prominent college fests.

Highlights include staging major pro-nights featuring renowned performers such as **Salim-Sulaiman**, **Amit Trivedi**, **Antara Mitra**, and DJ acts like **Teri Miko**, each drawing huge crowds and setting the fest's energy high.

The fest is designed to engage *students, artists, technophiles, and cultural enthusiasts* in an enriching mix of experiences—competitions like “Battle of Bands”, “Rampage” (fashion show), “Footloose” (contemporary dance), “Stage Play”, and “Cynosure” (main celebrity night); tech challenges like “Tech Hunt”, “Battledrome” (eSports), “Math-ify”; literary competitions such as “Shailee” (poetry & creative writing), “Anweshan” (research papers), “Samwaad” (parliamentary debate); creative zones like “CosCon” (cosplay), “Conceive” (photography), plus genre-spanning showcases in traditional dance, mime, street play (“Nukkad Natak”), and solo performances (“Mono Acte”).

Beyond entertainment, Synapse is a student-owned endeavor that empowers hundreds of volunteers and committee members across multiple teams—event management, hospitality, PR, design, sponsorship, finance, and web—each coordinating tasks, publicity, sponsorship outreach, and infrastructure to bring the fest to life.

The architecture of the fest is intentional: performances coexist with competitions, cultural expressions meet technological innovation, and student leadership combines with professional coordination—all under one dynamic roof. This complex orchestration needs a robust backend system to manage events, artists, ticketing, vendors, accommodation, schedules, sponsors, and reporting.

REFERENCES: [elections.daiict.ac.inKnowafest](https://elections.daiict.ac.in/Knowafest)

[UnstopKnowafestelections.daiict.ac.in](https://elections.daiict.ac.in/UnstopKnowafest)

[UnstopKnowafestFacebookUnstop](https://www.facebook.com/UnstopKnowafest)

2. Fact-Finding and Requirements Collection

2.1 BACKGROUND READING

2.1.1 “Synapse is a celebration...” — DA-IICT Annual Festival Committee Page

Source: Official description of Synapse on the DA-IICT committee site
[DA-IICT Elections](#)

What I Learned:

- Synapse is the annual techno-cultural festival at DA-IICT, described as “a celebration of culture, technology and life”.
- Organized fully by student committee teams handling: Event Management, Hospitality/Accommodation, Volunteers, Sponsorship, Public Relations, Design, and Web Design.
- Sponsorships majorly fund the fest—some budget is allocated to the SBG.

2.2 Interview

2.2.1 interview 1:

SYNAPSE: Interview plan

Participants : Dev Diyora

Ashish Kumar

Harsh Shah

Date : 02/08/2025

Place : cafeteria : **Time** : 16:00 : **Duration**: 05 minutes

To gain an understanding of the responsibilities and work of a synapse committee member.

interviewer(p1): how work is done as a committee?

Synapse committee member(p2): different members are assigned different work in groups of varying size, some may be required to sell, advertise tickets, some go to different colleges for advertising synapse, some work on room allotments, etc.

p1: Do you face any challenges in it?

P2 : sometimes it becomes difficult to find out who is responsible for which job.

P1: any other challenges?

P2 : during peak hours, when a lot of tickets are being sold or many events, accommodations,etc are being added, the database sometimes crashes.

CONCLUSIONS:

A record of committee members is required where who is responsible for which job can be recorded.

The database must be able to handle load without crashing.

2.2.2 interview 2:

Participants : Dev Diyora

Ashish Kumar

Trisha

Date : 02/08/2025

Place :Cafeteria : **Time** : 20:00 : **Duration** : 16 minutes

Purpose : To gain an understanding of the responsibilities and work of a Synapse committee member.

interviewer(p1): you have worked in synapse, what can you tell me about it.

Synapse committee member(p2): there are many events which take place in synapse, i personally worked in conducting the quizzing event. It was a theme based event with a harry-potter theme. The gifts for the winners were provided by sponsors. We had planned to conduct 2 levels of quizzes. If there were a draw between any groups we would have looked at the time taken by each group and decided the winner based on it.

There are many such events which take place in Synapse.

p1: Do you remember any challenges you faced in Synapse?

P2 : some members are stationed on the gate for registering on-spot registrations. It becomes hectic as there are many people coming on event day.

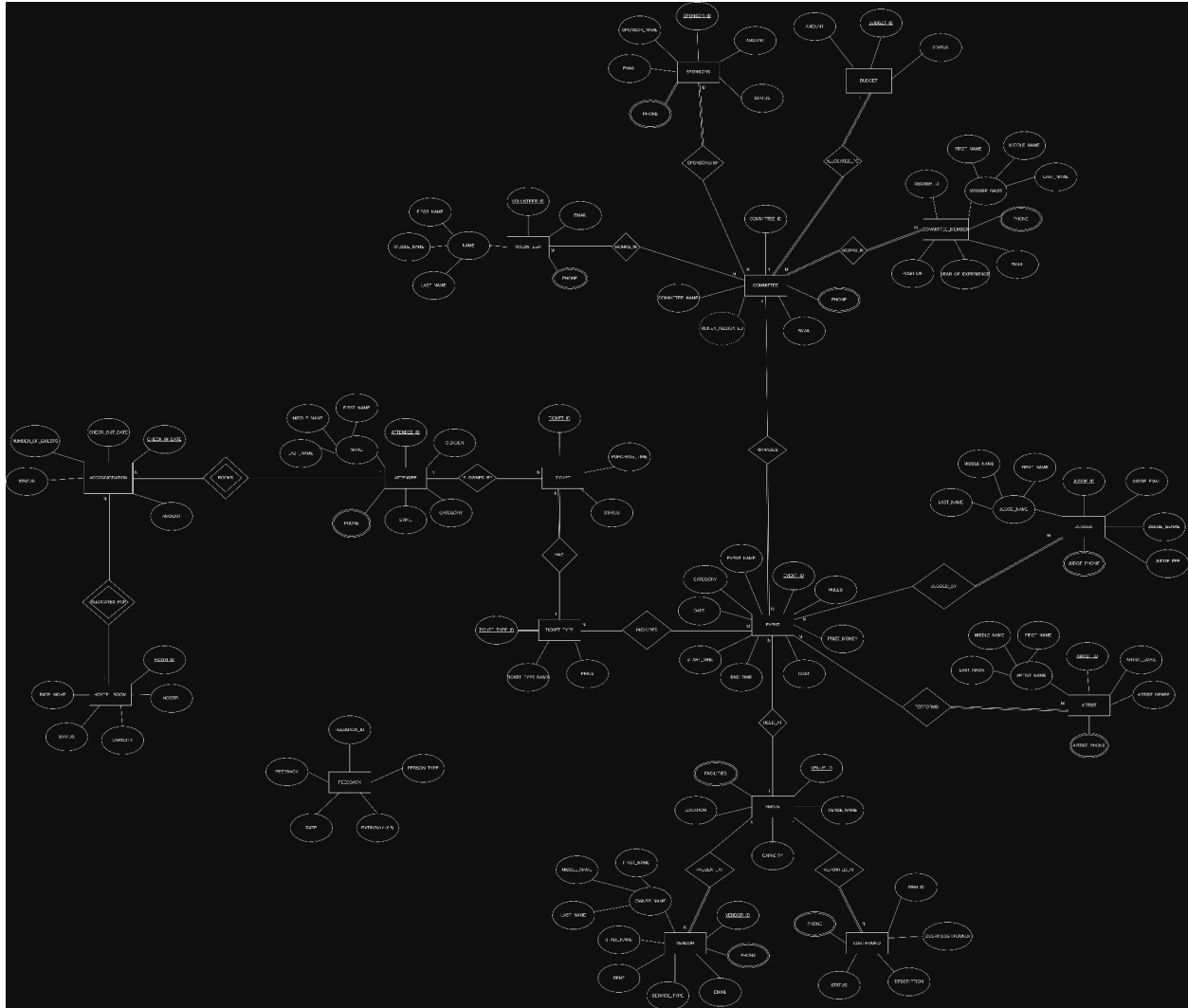
CONCLUSIONS:

Different events must not be mixed, their information must be stored separately, on the spot registrations can't be closed as maximum participation is encouraged, online registration must be encouraged as much as possible.

Also different sponsor's data must be stored.

ER DIAGRAM LINK

SYNAPSE_ER



Event subsystem

- EVENT
- JUDGES
- ARTIST
- VENUE

Ticketing subsystem

- ATTENDEE
- TICKET
- TICKET_TYPE

Accommodation subsystem

- ACCOMMODATION
- HOSTEL_ROOM

Finance subsystem

- BUDGET
- SPONSOR
- COMMITTEE

Staffing subsystem

- COMMITTEE_MEMBER
- VOLUNTEER

ERD TO RELATIONAL, DATA REDUNDANCY AND NORMALIZATION

TABLES:

EVENT(event_id,event_name,category,rules,prize_money,date,start_time,end_time,cost_event,venue_id,comm_id)

JUDGES(judge_id,first_name,middle_name,last_name,email,fee,genre)

JUDGES_PHONE(judge_id,phone)

EVENT_JUDGES(event_id,judge_id)

ARTIST(artist_id,first_name,middle_name,last_name,email,fee,genre)

ARTIST_PHONE(artist_id,phone)

EVENT_ARTIST(event_id,artist_id)

VENUE(venue_id,venue_name,capacity,location)

VENUE_FACILITY(venue_id,facility) //AC,WASHROOM,etc

COMMITTEE(comm_id,comm_name,email)

COMMITTEE_PHONE(comm_id,phone)

TICKET_TYPE(ticket_type_id,ticket_type_name,price)

TICKET_TYPE_EVENT(ticket_type_id,event_id)

TICKET(ticket_id,purchase_time,status,attendee_id,ticket_type_id)

ATTENDEE(attendee_id,first_name,middle_name,last_name,gender,category,email,dob)
//student,prof,etc

ATTENDEE_PHONE(attendee_id,phone)

ACCOMMODATION(attendee_id,room_id,check_in_date_time,check_out_date,number_of_guests,amount,status)

HOSTEL_ROOM(room_id,hostel_name,capacity,status,rate_per_night)

VENDOR(vendor_id,venue_id,stall_name,first_name,middle_name,last_name,email,service_type,rent_per_day)

VENDOR_PHONE(vendor_id,phone)

LOST_FOUND(item_id,query(lost/found),description,status,venue_id)

LOST_FOUND_PHONE(item_id,phone)

BUDGET(budget_id,comm_id,amount,status)

SPONSOR(sponsor_id,sponsor_name,email)

COMMITTEE_SPONSOR(comm_id,sponsor_id,amount,status)

SPONSOR_PHONE(sponsor_id,phone)

COMMITTEE_MEMBER(member_id,first_name,middle_name,last_name,email,position,year_experience)

COMMITTEE_MEMBER_PHONE(member_id,phone)

COMMITTEE_COMMITTEE_MEMBER(comm_id,member_id)

VOLUNTEER(volunteer_id,first_name,middle_name,last_name,email)

VOLUNTEER_PHONE(volunteer_id,phone)

COMMITTEE_VOLUNTEER(comm_id,volunteer_id)

FEEDBACK(feedback_id,feedback,reviewer_type,rating(out of 5),date)

FUNCTIONAL DEPENDENCIES

EVENT

event_id→event_name, category, rules, prize_money, date, start_time, end_time, cost_event, venue_id, comm_id

JUDGES

judge_id→first_name, middle_name, last_name, email, fee, genre

JUDGES_PHONE

(judge_id, phone)→all attributes

EVENT_JUDGES

(event_id, judge_id)→all attributes

ARTIST

artist_id→first_name,middle_name,last_name,email,fee,genre

ARTIST_PHONE

(artist_id,phone)→all attributes

EVENT_ARTIST

(event_id,artist_id)→all attributes

VENUE

venue_id→venue_name,capacity,location

VENUE_FACILITY

(venue_id,facility)→all attributes

COMMITTEE

comm_id→comm_name,email

COMMITTEE_PHONE

(comm_id,phone)→all attributes

TICKET_TYPE

ticket_type_id→ticket_type_name,price

TICKET_TYPE_EVENT

(ticket_type_id,event_id)→all attributes

TICKET

ticket_id→purchase_time,status,attendee_id,ticket_type_id

ATTENDEE

attendee_id→first_name,middle_name,last_name,gender,category,email

ATTENDEE_PHONE

(attendee_id,phone)→all attributes

ACCOMODATION

(attendee_id,room_id,check_in_date_time)→check_out_date,number_of_guests,amount,status

HOSTEL_ROOM

room_id→hostel_name,capacity,status,rate_per_night

VENDOR

vendor_id→venue_id, stall_name, first_name, middle_name, last_name, email, service_type, rent_per_day

VENDOR_PHONE

(vendor_id, phone)→all attributes

LOST_FOUND

item_id→query, description, status, venue_id

LOST_FOUND_PHONE

(item_id, phone)→all attributes

BUDGET

budget_id→comm_id, amount, status

SPONSOR

sponsor_id→sponsor_name, email

COMMITTEE_SPONSOR

(comm_id, sponsor_id)→amount, status

SPONSOR_PHONE

(sponsor_id, phone)→all attributes

COMMITTEE_MEMBER

member_id→first_name, middle_name, last_name, email, position, year_experience

COMMITTEE_MEMBER_PHONE

(member_id, phone)→all attributes

COMMITTEE_COMMITTEE_MEMBER

(comm_id, member_id)→all attributes

VOLUNTEER

volunteer_id→first_name, middle_name, last_name, email

VOLUNTEER_PHONE

(volunteer_id, phone)→all attributes

COMMITTEE_VOLUNTEER

(comm_id, volunteer_id)→all attributes

FEEDBACK

feedback_id→feedback, person_type, rating, date

NORMALIZATION:

1NF:

All attributes are single valued.
So all tables are already in 1 NF.

2NF:

Relations Already in 2NF

Relations having single-attribute primary keys are automatically in 2NF.

Examples:

EVENT
VENUE
COMMITTEE
ATTENDEE
TICKET
HOSTEL_ROOM
SPONSOR
BUDGET
VOLUNTEER
COMMITTEE_MEMBER

because all non-key attributes depend entirely on their primary key.

Composite Key Relations Analysis

We now analyze relations with composite keys.

1. COMMITTEE_SPONSOR

Relation:

**COMMITTEE_SPONSOR(
comm_id,
sponsor_id,
amount,
status
)**

Primary Key:

(comm_id, sponsor_id)

FD:

(comm_id,sponsor_id)→amount,status

Neither:

comm_id→amount

nor

Sponsor_id→amount

Similarly for status.

Therefore:

- no partial dependency exists
- relation is in 2NF

2. ACCOMMODATION

Relation:

**ACCOMMODATION(
attendee_id,
room_id,
check_in_date_time,
check_out_date,
number_of_guests,
amount,**

status
)

Primary Key:

(attendee_id, room_id, check_in_date_time)

FD:

(attendee_id, room_id, check_in_date_time) → check_out_date, number_of_guests, amount, status

No attribute depends only on:

- attendee_id
- Room_id
- check_in_date_time
- or partial subset

Hence:

- no partial dependency
- relation is in 2NF

3. EVENT_ARTIST

Initial relation:

EVENT_ARTIST(event_id, artist_id)

PK: (event_id, artist_id)

FD: (event_id, artist_id) → (event_id, artist_id)

Cannot determine artist_id just from event_id and vice versa

Hence already in 2NF.

4. EVENT_JUDGES

Initial relation:

EVENT_JUDGES(event_id, judge_id)

Just like event_artist.

Also already in 2NF.

Conclusion for 2NF

All relations satisfy 2NF because no non-prime attribute depends on only part of a composite key

3NF and other improvements:

ISSUE 1 — ARTIST relation

FD:

$\text{artist_id} \rightarrow \text{fee}$

Problem:

Artist fee is not intrinsic to artist.

The fee depends on which event artist performs in

Thus:

$(\text{event_id}, \text{artist_id}) \rightarrow \text{fee}$ is the correct dependency.

Decomposition

Final Relations

ARTIST(artist_id,first_name,middle_name,last_name,email,genre)

and

EVENT_ARTIST(event_id,artist_id,fee)

Justification

This removes semantic redundancy and update anomalies.

Example problem before normalization:

If same artist performs in 2 events with different fees:

Artist A $\rightarrow$ ₹50,000 in event 1

Artist A → ₹1,00,000 in event 3

Single fee attribute becomes inconsistent.

Thus decomposition improves consistency.

ISSUE 2 — JUDGES relation

Same issue as of artist table.

Decomposition

Final Relations

JUDGES(judge_id,first_name,middle_name,last_name,email,genre)

and

EVENT_JUDGES(event_id,judge_id,fee)

Justification

This avoids inconsistent judge fees across events

ISSUE 3 — VENUE_FACILITIES

I want some fix values for facilities so i will create a new table facility

facility(facility_id,facility_name)

And venue facility becomes(venue_id,facility_id)

Other Relations

Remaining relations already satisfy 3NF because:

- all non-key attributes depend only on candidate key
- no transitive dependency exists

FINAL TABLES:

EVENT(event_id, event_name, category, rules, prize_money, date, start_time, end_time, cost_event, venue_id, comm_id)

JUDGES(judge_id, first_name, middle_name, last_name, email, genre)

JUDGES_PHONE(judge_id, phone)

EVENT_JUDGES(event_id, judge_id, fee)

ARTIST(artist_id, first_name, middle_name, last_name, email, genre)

ARTIST_PHONE(artist_id, phone)

EVENT_ARTIST(event_id, artist_id, fee)

VENUE(venue_id, venue_name, capacity, location)

facility(facility_id, facility_name)

VENUE_FACILITY(venue_id, facility_id)

COMMITTEE(comm_id, comm_name, email)

COMMITTEE_PHONE(comm_id, phone)

TICKET_TYPE(ticket_type_id, ticket_type_name, price)

TICKET_TYPE_EVENT(ticket_type_id,event_id)

TICKET(ticket_id,purchase_time,status,attendee_id,ticket_type_id)

ATTENDEE(attendee_id,first_name,middle_name,last_name,gender,category,email,dob)
//student,prof,etc

ATTENDEE_PHONE(attendee_id,phone)

ACCOMMODATION(attendee_id,room_id,check_in_date_time,check_out_date,amount,status)

HOSTEL_ROOM(room_id,hostel_name,capacity,,rate_per_night)

VENDOR(vendor_id,venue_id,stall_name,first_name,middle_name,last_name,email,service_type,rent_per_day)

VENDOR_PHONE(vendor_id,phone)

LOST_FOUND(item_id,report_type,description,status,phone,venue_id)

BUDGET(budget_id,comm_id,amount,status)

SPONSOR(sponsor_id,sponsor_name,email)

COMMITTEE_SPONSOR(comm_id,sponsor_id,amount,status)

SPONSOR_PHONE(sponsor_id,phone)

COMMITTEE_MEMBER(member_id,first_name,middle_name,last_name,email,position,year_experience)

COMMITTEE_MEMBER_PHONE(member_id,phone)

COMMITTEE_COMMITTEE_MEMBER(comm_id,member_id)

VOLUNTEER(volunteer_id,first_name,middle_name,last_name,email)

VOLUNTEER_PHONE(volunteer_id,phone)

COMMITTEE_VOLUNTEER(comm_id,volunteer_id)

FEEDBACK(feedback_id,feedback,reviewer_type,rating(out of 5),date)

DDL STATEMENTS:

```
CREATE TABLE committee (  
    comm_id SERIAL PRIMARY KEY,  
    comm_name VARCHAR(100) NOT NULL,  
    email VARCHAR(255) UNIQUE NOT NULL  
);
```

```
CREATE TABLE committee_phone (  
    comm_id INT NOT NULL,  
    phone VARCHAR(15) NOT NULL,  
  
    PRIMARY KEY (comm_id, phone),  
  
    FOREIGN KEY (comm_id)  
        REFERENCES committee(comm_id)  
        ON DELETE CASCADE  
);
```

```
CREATE TABLE venue (  
    venue_id SERIAL PRIMARY KEY,  
    venue_name VARCHAR(150) NOT NULL,  
    capacity INT NOT NULL CHECK (capacity > 0),  
    location VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE facility (  
    facility_id SERIAL PRIMARY KEY,  
  
    facility_name VARCHAR(50) NOT NULL UNIQUE  
);
```

```
CREATE TABLE venue_facility (  
    venue_id INT NOT NULL,  
    facility_id INT NOT NULL,  
  
    PRIMARY KEY (venue_id, facility_id),
```

```
FOREIGN KEY (venue_id)
REFERENCES venue(venue_id)
ON DELETE CASCADE,

FOREIGN KEY (facility_id)
REFERENCES facility(facility_id)
ON DELETE RESTRICT
);
```

```
CREATE TABLE committee_member (
  member_id SERIAL PRIMARY KEY,

  first_name VARCHAR(50) NOT NULL,
  middle_name VARCHAR(50),
  last_name VARCHAR(50) NOT NULL,

  email VARCHAR(255) UNIQUE NOT NULL,

  position VARCHAR(100) NOT NULL,

  year_experience INT NOT NULL
  CHECK (year_experience >= 0)
);
```

```
CREATE TABLE committee_member_phone (
  member_id INT NOT NULL,
  phone VARCHAR(15) NOT NULL,

  PRIMARY KEY (member_id, phone),

  FOREIGN KEY (member_id)
  REFERENCES committee_member(member_id)
  ON DELETE CASCADE
);
```

```
CREATE TABLE committee_committee_member (
  comm_id INT NOT NULL,
  member_id INT NOT NULL,

  PRIMARY KEY (comm_id, member_id),
```

```
FOREIGN KEY (comm_id)
  REFERENCES committee(comm_id)
  ON DELETE CASCADE,

FOREIGN KEY (member_id)
  REFERENCES committee_member(member_id)
  ON DELETE CASCADE
);
```

```
CREATE TABLE volunteer (
  volunteer_id SERIAL PRIMARY KEY,

  first_name VARCHAR(50) NOT NULL,
  middle_name VARCHAR(50),
  last_name VARCHAR(50) NOT NULL,

  email VARCHAR(255) UNIQUE NOT NULL
);
```

```
CREATE TABLE volunteer_phone (
  volunteer_id INT NOT NULL,
  phone VARCHAR(15) NOT NULL,

  PRIMARY KEY (volunteer_id, phone),

  FOREIGN KEY (volunteer_id)
    REFERENCES volunteer(volunteer_id)
    ON DELETE CASCADE
);
```

```
CREATE TABLE committee_volunteer (
  comm_id INT NOT NULL,
  volunteer_id INT NOT NULL,

  PRIMARY KEY (comm_id, volunteer_id),

  FOREIGN KEY (comm_id)
    REFERENCES committee(comm_id)
    ON DELETE CASCADE,

  FOREIGN KEY (volunteer_id)
```



```
REFERENCES volunteer(volunteer_id)
ON DELETE CASCADE
);
```

```
CREATE TABLE budget (
  budget_id SERIAL PRIMARY KEY,

  comm_id INT NOT NULL UNIQUE,

  amount NUMERIC(12,2) NOT NULL
  CHECK (amount >= 0),

  status VARCHAR(20) NOT NULL
  CHECK (status IN ('PENDING', 'APPROVED')),

  FOREIGN KEY (comm_id)
  REFERENCES committee(comm_id)
  ON DELETE RESTRICT
);
```

```
CREATE TABLE sponsor (
  sponsor_id SERIAL PRIMARY KEY,

  sponsor_name VARCHAR(150) NOT NULL,

  email VARCHAR(255) UNIQUE NOT NULL
);
```

```
CREATE TABLE sponsor_phone (
  sponsor_id INT NOT NULL,
  phone VARCHAR(15) NOT NULL,

  PRIMARY KEY (sponsor_id, phone),

  FOREIGN KEY (sponsor_id)
  REFERENCES sponsor(sponsor_id)
  ON DELETE CASCADE
);
```

```
CREATE TABLE committee_sponsor (
```

```

comm_id INT NOT NULL,
sponsor_id INT NOT NULL,

amount NUMERIC(12,2) NOT NULL
    CHECK (amount > 0),

status VARCHAR(20) NOT NULL
    CHECK (status IN ('PENDING', 'APPROVED')),

PRIMARY KEY (comm_id, sponsor_id),

FOREIGN KEY (comm_id)
    REFERENCES committee(comm_id)
    ON DELETE RESTRICT,

FOREIGN KEY (sponsor_id)
    REFERENCES sponsor(sponsor_id)
    ON DELETE RESTRICT
);

```

```

CREATE TABLE event (
    event_id SERIAL PRIMARY KEY,

    event_name VARCHAR(150) NOT NULL UNIQUE,

    category VARCHAR(50) NOT NULL
        CHECK (
            category IN (
                'CODING',
                'STREET_PLAY',
                'STANDUP',
                'DJ',
                'DANCE',
                'MUSIC',
                'TALENT'
            )
        ),

    rules TEXT,

    prize_money NUMERIC(12,2) NOT NULL
        CHECK (prize_money >= 0),

```

```

date DATE NOT NULL,

start_time TIME NOT NULL,

end_time TIME NOT NULL,

cost_event NUMERIC(12,2) NOT NULL
    CHECK (cost_event >= 0),

venue_id INT NOT NULL,

comm_id INT NOT NULL,

CHECK (start_time < end_time),

FOREIGN KEY (venue_id)
    REFERENCES venue(venue_id)
    ON DELETE RESTRICT,

FOREIGN KEY (comm_id)
    REFERENCES committee(comm_id)
    ON DELETE RESTRICT
);

CREATE TABLE judges (
    judge_id SERIAL PRIMARY KEY,

    first_name VARCHAR(50) NOT NULL,
    middle_name VARCHAR(50),
    last_name VARCHAR(50) NOT NULL,

    email VARCHAR(255) UNIQUE NOT NULL,

    genre VARCHAR(50) NOT NULL
    CHECK (
        genre IN (
            'CODING',
            'STREET_PLAY',
            'STANDUP',
            'DJ',
            'DANCE',
            'MUSIC',
            'TALENT'

```

```
)  
)  
);
```

```
CREATE TABLE judges_phone (  
    judge_id INT NOT NULL,  
    phone VARCHAR(15) NOT NULL,  
  
    PRIMARY KEY (judge_id, phone),  
  
    FOREIGN KEY (judge_id)  
        REFERENCES judges(judge_id)  
        ON DELETE CASCADE  
);
```

```
CREATE TABLE event_judges (  
    event_id INT NOT NULL,  
    judge_id INT NOT NULL,  
  
    fee NUMERIC(12,2) NOT NULL  
        CHECK (fee >= 0),  
  
    PRIMARY KEY (event_id, judge_id),  
  
    FOREIGN KEY (event_id)  
        REFERENCES event(event_id)  
        ON DELETE RESTRICT,  
  
    FOREIGN KEY (judge_id)  
        REFERENCES judges(judge_id)  
        ON DELETE RESTRICT  
);
```

```
CREATE TABLE artist (  
    artist_id SERIAL PRIMARY KEY,  
  
    first_name VARCHAR(50) NOT NULL,  
    middle_name VARCHAR(50),  
    last_name VARCHAR(50) NOT NULL,  
  
    email VARCHAR(255) UNIQUE NOT NULL,
```

```
genre VARCHAR(50) NOT NULL
CHECK (
    genre IN (
        'CODING',
        'STREET_PLAY',
        'STANDUP',
        'DJ',
        'DANCE',
        'MUSIC',
        'TALENT'
    )
)
);
```

```
CREATE TABLE artist_phone (
    artist_id INT NOT NULL,
    phone VARCHAR(15) NOT NULL,

    PRIMARY KEY (artist_id, phone),

    FOREIGN KEY (artist_id)
        REFERENCES artist(artist_id)
        ON DELETE CASCADE
);
```

```
CREATE TABLE event_artist (
    event_id INT NOT NULL,
    artist_id INT NOT NULL,

    fee NUMERIC(12,2) NOT NULL
    CHECK (fee >= 0),

    PRIMARY KEY (event_id, artist_id),

    FOREIGN KEY (event_id)
        REFERENCES event(event_id)
        ON DELETE RESTRICT,

    FOREIGN KEY (artist_id)
        REFERENCES artist(artist_id)
        ON DELETE RESTRICT
);
```

);

```
CREATE TABLE attendee (  
  attendee_id SERIAL PRIMARY KEY,  
  
  first_name VARCHAR(50) NOT NULL,  
  middle_name VARCHAR(50),  
  last_name VARCHAR(50) NOT NULL,  
  
  gender VARCHAR(10) NOT NULL  
  CHECK (  
    gender IN (  
      'MALE',  
      'FEMALE',  
      'OTHER'  
    )  
  ),  
  
  category VARCHAR(30) NOT NULL  
  CHECK (  
    category IN (  
      'STUDENT',  
      'PROFESSOR',  
      'WORKING_PROFESSIONAL',  
      'OUTSIDER'  
    )  
  ),  
  
  email VARCHAR(255) UNIQUE NOT NULL,  
  
  dob DATE NOT NULL  
);
```

```
CREATE TABLE attendee_phone (  
  attendee_id INT NOT NULL,  
  phone VARCHAR(15) NOT NULL,  
  
  PRIMARY KEY (attendee_id, phone),  
  
  FOREIGN KEY (attendee_id)  
    REFERENCES attendee(attendee_id)  
    ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE ticket_type (  
    ticket_type_id SERIAL PRIMARY KEY,  
  
    ticket_type_name VARCHAR(100) NOT NULL,  
  
    price NUMERIC(12,2) NOT NULL  
        CHECK (price >= 0)  
);
```

```
CREATE TABLE ticket_type_event (  
    ticket_type_id INT NOT NULL,  
    event_id INT NOT NULL,  
  
    PRIMARY KEY (ticket_type_id, event_id),  
  
    FOREIGN KEY (ticket_type_id)  
        REFERENCES ticket_type(ticket_type_id)  
        ON DELETE RESTRICT,  
  
    FOREIGN KEY (event_id)  
        REFERENCES event(event_id)  
        ON DELETE RESTRICT  
);
```

```
CREATE TABLE ticket (  
    ticket_id SERIAL PRIMARY KEY,  
  
    purchase_time TIMESTAMP NOT NULL  
        DEFAULT CURRENT_TIMESTAMP,  
  
    status VARCHAR(10) NOT NULL  
        CHECK (  
            status IN (  
                'ACTIVE',  
                'USED'  
            )  
        ),  
  
    attendee_id INT NOT NULL,
```

```
ticket_type_id INT NOT NULL,  
  
FOREIGN KEY (attendee_id)  
  REFERENCES attendee(attendee_id)  
  ON DELETE RESTRICT,  
  
FOREIGN KEY (ticket_type_id)  
  REFERENCES ticket_type(ticket_type_id)  
  ON DELETE RESTRICT  
);
```

```
CREATE TABLE hostel_room (  
  room_id VARCHAR(20) PRIMARY KEY,  
  
  hostel_name VARCHAR(100) NOT NULL,  
  
  capacity INT NOT NULL  
    CHECK (capacity > 0),  
  
  rate_per_night NUMERIC(12,2) NOT NULL  
    CHECK (rate_per_night >= 0)  
);
```

```
CREATE TABLE accommodation (  
  attendee_id INT NOT NULL,  
  
  room_id INT NOT NULL,  
  
  check_in_date_time TIMESTAMP NOT NULL,  
  
  check_out_date TIMESTAMP NOT NULL,  
  
  amount NUMERIC(12,2) NOT NULL  
    CHECK (amount >= 0),  
  
  status VARCHAR(20) NOT NULL  
    CHECK (  
      status IN (  
        'ACTIVE',  
        'COMPLETED'
```



```

    )
),

PRIMARY KEY (
    attendee_id,
    room_id,
    check_in_date_time
),

CHECK (
    check_out_date > check_in_date_time
),

FOREIGN KEY (attendee_id)
REFERENCES attendee(attendee_id)
ON DELETE RESTRICT,

FOREIGN KEY (room_id)
REFERENCES hostel_room(room_id)
ON DELETE RESTRICT
);

CREATE TABLE vendor (
    vendor_id SERIAL PRIMARY KEY,

    venue_id INT NOT NULL,

    stall_name VARCHAR(100) NOT NULL,

    first_name VARCHAR(50) NOT NULL,
    middle_name VARCHAR(50),
    last_name VARCHAR(50) NOT NULL,

    email VARCHAR(255) UNIQUE NOT NULL,

    service_type VARCHAR(20) NOT NULL
    CHECK (
        service_type IN (
            'FOOD',
            'BEVERAGES',
            'MERCHANDISE',
            'BOOKS',
            'TECHNOLOGY',

```

```

        'GAMES',
        'OTHER'
    )
),

rent_per_day NUMERIC(12,2) NOT NULL
    CHECK (rent_per_day >= 0),

FOREIGN KEY (venue_id)
    REFERENCES venue(venue_id)
    ON DELETE RESTRICT
);

```

```

CREATE TABLE vendor_phone (
    vendor_id INT NOT NULL,
    phone VARCHAR(15) NOT NULL,

    PRIMARY KEY (vendor_id, phone),

    FOREIGN KEY (vendor_id)
        REFERENCES vendor(vendor_id)
        ON DELETE CASCADE
);

```

```

CREATE TABLE lost_found (
    item_id SERIAL PRIMARY KEY,

    report_type VARCHAR(10) NOT NULL
        CHECK (
            report_type IN (
                'LOST',
                'FOUND'
            )
        ),

    description TEXT NOT NULL,

    phone VARCHAR(15) NOT NULL,

    status VARCHAR(20) NOT NULL
        CHECK (

```

```

        status IN (
            'OPEN',
            'RESOLVED'
        )
    ),

venue_id INT NOT NULL,

FOREIGN KEY (venue_id)
    REFERENCES venue(venue_id)
    ON DELETE RESTRICT
);

CREATE TABLE feedback (
    feedback_id SERIAL PRIMARY KEY,

    feedback TEXT NOT NULL,

    reviewer_type VARCHAR(30) NOT NULL
        CHECK (
            reviewer_type IN (
                'ATTENDEE',
                'ARTIST',
                'VENDOR',
                'VOLUNTEER',
                'JUDGE',
                'COMMITTEE_MEMBER'
            )
        ),

    rating INT NOT NULL
        CHECK (
            rating BETWEEN 1 AND 5
        ),

    feedback_date DATE NOT NULL
        DEFAULT CURRENT_DATE
);

```

TRIGGERS

Trigger 1: ACCOMODATION AMOUNT DERIVATION

```
CREATE OR REPLACE FUNCTION calculate_accommodation_amount()  
  
RETURNS TRIGGER AS $$  
  
DECLARE  
  
    room_rate NUMERIC(12,2);  
  
    num_nights INT;  
  
BEGIN  
  
  
    SELECT rate_per_night  
  
    INTO room_rate  
  
    FROM hostel_room  
  
    WHERE room_id = NEW.room_id;  
  
  
    num_nights :=  
  
        NEW.check_out_date::date  
  
        - NEW.check_in_date_time::date;  
  
  
    IF num_nights <= 0 THEN  
  
        RAISE EXCEPTION  
  
        'Accommodation must be booked for at least one night';  
  
    END IF;
```

```
NEW.amount := room_rate * num_nights;

RETURN NEW;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_calculate_accommodation_amount
BEFORE INSERT OR UPDATE
ON accommodation
FOR EACH ROW
EXECUTE FUNCTION calculate_accommodation_amount();
```

Trigger 2: Room Availability / Date Overlap

```
CREATE OR REPLACE FUNCTION check_room_availability()
RETURNS TRIGGER AS
$$
DECLARE
    room_cap INT;
```

```
current_occupancy INT;

BEGIN

    SELECT capacity
    INTO room_cap
    FROM hostel_room
    WHERE room_id = NEW.room_id;

    SELECT COUNT(*)
    INTO current_occupancy
    FROM accommodation a
    WHERE a.room_id = NEW.room_id

        AND a.status = 'ACTIVE'

        AND NEW.check_in_date_time < a.check_out_date
        AND NEW.check_out_date > a.check_in_date_time;

    IF TG_OP = 'UPDATE' THEN

        current_occupancy := current_occupancy - 1;

    END IF;

    IF current_occupancy >= room_cap THEN

        RAISE EXCEPTION
```

```

        'Room % is full for selected dates',

        NEW.room_id;

    END IF;


    RETURN NEW;

END;

$$ LANGUAGE plpgsql;


DROP TRIGGER IF EXISTS trg_check_room_availability
ON accommodation;


CREATE TRIGGER trg_check_room_availability
BEFORE INSERT OR UPDATE
ON accommodation
FOR EACH ROW
EXECUTE FUNCTION check_room_availability();

```

Trigger 3: Venue Schedule Conflict

```

CREATE OR REPLACE FUNCTION check_venue_schedule()

```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    IF EXISTS (
```

```
        SELECT 1
```

```
        FROM event e
```

```
        WHERE e.venue_id = NEW.venue_id
```

```
            AND e.date = NEW.date
```

```
            AND e.event_id <> COALESCE(NEW.event_id,-1)
```

```
            AND NEW.start_time < e.end_time
```

```
            AND NEW.end_time > e.start_time
```

```
    )
```

```
    THEN
```

```
        RAISE EXCEPTION
```

```
        'Venue already occupied during selected time slot';
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```



```
CREATE TRIGGER trg_check_venue_schedule  
BEFORE INSERT OR UPDATE  
ON event  
FOR EACH ROW  
EXECUTE FUNCTION check_venue_schedule();
```

